Python Basics

Premium Study Notes

Notebook: 01_Python_Basics.ipynb

Topics: Introduction | Data Types | Variables | print() Variants | Operators | Input | Typecasting | Comments

Table of Contents

1. **Agenda & Module Overview**
2. **print() Function**
   -> Built-in Functions
   -> Basic print() Examples
3. **Variables & Constants**
4. **Comments in Python**
   -> Single-line Comments
   -> Multi-line Comments
5. **Data Types in Python**
   -> int float str bool
   -> Checking Data Types
6. **Variable Naming Best Practices**
7. **print() Variants - 4 Ways**
   -> Way 1: Basic
   -> Way 2: Comma-separated
   -> Way 3: format()
   -> Way 4: f-strings
8. **Printing Curly Braces with f-strings**
9. **input() Function**
   -> Type of Input
   -> Type Conversion of Input
10. **round() Function**
11. **Typecasting**
    -> Implicit Typecasting
    -> Explicit Typecasting
12. **Operators**
    -> Arithmetic
    -> Assignment
    -> Comparison
    -> Logical
    -> Membership
    -> Identity
13. **Memory & id() Function**
14. **To-Do Tasks (5 Exercises)**

1 Agenda for our Python Module

1.	Introduction to Python
2.	Data types and variables
3.	print() function variants
4.	Operators
5.	Conditional statements (if, elif, else statements)
6.	Loops (for and while loops)
7.	Data structures (lists, tuples, sets, dictionaries)
8.	Functions
9.	Exception handling (try, except blocks)
10.	File handling (reading and writing files)
11.	Modules and packages
12.	Object-oriented programming (classes and objects)

2 print() Function

What is a built-in function?

A built-in function is pre-defined in Python and can be used without importing any libraries.
Examples: print(), len(), type(), int(), str(), float(), list(), dict(), set(), tuple(), range(), input()

Code

```
print("Hello, World!") # prints Hello, World!

print(42) # prints 42

print(3.14) # prints 3.14

print(True) # prints True

print(None) # prints None

print(len("Hello")) # prints 5

print(type(42)) # prints <class 'int'>
```

Output

```
Hello, World!

42

3.14
```

```
True

None

5

<class 'int'>
```

3 Variables & Constants

Definition

A **variable** is a named container for storing data. A **constant** is the actual value stored in it. Example: in `employee_name = "Dr. Darshan Ingle"`, the variable is `employee_name` and the constant is `"Dr. Darshan Ingle"`.

Code

```
employee_name = "Dr. Darshan Ingle"

employee_age = 37

employee_salary = 75000.50

employee_is_manager = True

employee_department = "Banking and Finance"

print(employee_name)

print(employee_age)

print(employee_salary)

print(employee_is_manager)

print(employee_department)
```

Output

```
Dr. Darshan Ingle

37

75000.5

True

Banking and Finance
```

4 Comments in Python

What is a comment?

A comment is text in a program ignored by the Python interpreter. Used to explain code and make it more readable. Comments start with the # symbol.

Type	Syntax	Example
Single-line	Start with #	`# This is a comment`
Multi-line	Triple quotes (''' or """)	`''' Multi-line comment '''`

Code

```
# This is a single-line comment

# print('Hello') # commented-out code

'''

This is a multi-line comment

that spans multiple lines.

'''
```

5 Data Types in Python

#	Type	Description	Examples
1.	**int**	Integer - whole numbers	`1, 2, -4, 0`
2.	**float**	Floating-point - decimal numbers	`3.14, -0.5, 2.0`
3.	**str**	String - text data	`"Hello", 'Python'`
4.	**bool**	Boolean - truth values	`True, False`

Note

Use the **type()** function to check the datatype of any variable or value.

Code

```
units_sold = 150

print(type(units_sold)) # int

price_per_unit = 19.99

print(type(price_per_unit)) # float

product_name = "Wireless Mouse"

print(type(product_name)) # str

is_available = True
```

```
print(type(is_available)) # bool
```

Output

```
<class 'int'>

<class 'float'>

<class 'str'>

<class 'bool'>
```

6 Good Practices for Naming Variables

1. Use descriptive names: Choose names that clearly describe the variable's purpose

2. Use lowercase with underscores: e.g., employee_name, total_sales (snake_case)

3. Avoid reserved words: Don't use: if, else, for, while, def, return, True, False, None

4. Be consistent: Use a consistent naming convention throughout your code

5. Avoid special characters: Don't use @, #, $, %, &, * in variable names

6. Start with letter or underscore: Not a number - valid: _var, myVar; invalid: 1var

7. Keep it concise: Be descriptive but also try to keep names short

8. Use meaningful prefixes: e.g., is_ for booleans, num_ for numbers

9. Avoid unclear abbreviations: Write out words when possible for clarity

10. Comment complex variables: Add comments to explain complex calculations

Good naming examples:

Code

```
employee_name = "John Doe" # clear and descriptive

total_sales = 10000.50

is_active = True

num_items = 5
```

Bad naming examples:

Code

```
x = "John Doe" # unclear

a = 10000.50 # unclear

flag = True # vague

n = 5 # unclear
```

7 print() Function Variants - 4 Ways

Way 1 - Basic print()

Code

```
num1 = 10

num2 = 20

sum_result = num1 + num2

print(num1)

print(num2)

print(sum_result)
```

Output

```
10

20

30
```

Way 2 - Comma-separated print()

Each comma adds a space automatically between items.

Code

```
num1 = 10

num2 = 20

sum_result = num1 + num2

print("The total of", num1, "and", num2, "is", sum_result)
```

Output

```
The total of 10 and 20 is 30
```

Way 3 - format() Method

Use {} as placeholders, filled by .format() in order.

Code

```
num1 = 10

num2 = 20

sum_result = num1 + num2

print("The total of {} and {} is {}".format(num1, num2, sum_result))
```

Output

```
The total of 10 and 20 is 30
```

Way 4 - f-strings (formatted string literals)

Prefix string with f, embed variables directly inside {}. Most concise and readable.

Code

```
num1 = 10

num2 = 20

sum_result = num1 + num2

print(f"The total of {num1} and {num2} is {sum_result}")
```

Output

```
The total of 10 and 20 is 30
```

8 Printing Curly Braces with f-strings

Q: How to print literal {} braces in an f-string?

Answer: Use double curly braces {{ and }} to escape them in an f-string.

Code

```
value = 42

print(f"The value is {{ {value} }}")
```

Output

```
The value is { 42 }
```

9 Task: Area Calculation Exercise

Task

Write a program to calculate the area of a rectangle by accepting length and breadth.

Assume length as the radius of a circle and calculate area and perimeter of the circle.

Divide the area of rectangle by perimeter of circle and print using all 4 print() approaches.

10 input() Function - Taking User Input

What is input()?

The input() function allows you to take input from the user. It waits for the user to type and press Enter, then returns the input as a **string**.

Syntax: `input(prompt)` — prompt is optional text shown to user.

Code

```
name = input('Enter your name: ')

print(name)

print(type(name)) # always returns str
```

Output

```
Darshan

<class 'str'>
```

Important Note

The datatype of input() is **always string**, regardless of what the user types.

Type Conversion of Input

Convert to integer:

Code

```
name = int(input())

print(name)

print(type(name))
```

Output

```
180

<class 'int'>
```

Convert to float and do arithmetic:

Code

```
num1 = float(input('Enter first number: '))

num2 = float(input('Enter second number: '))

sum_result = num1 + num2

print(f"The total of {num1} and {num2} is {round(sum_result, 2)}")
```

Output

```
The total of 121.12 and 321.321 is 442.44
```

11 round() Function

Definition

round(number, ndigits) - rounds a float to the specified number of decimal places. ndigits defaults to 0.

Code

```
sum_result = 442.44100000000003

print(round(sum_result)) # nearest integer

print(round(sum_result, 2)) # 2 decimal places

print(round(sum_result, 3)) # 3 decimal places
```

Output

```
442

442.44

442.441
```

12 Typecasting

What is typecasting?

Typecasting is converting a variable from one datatype to another.

1. Implicit Typecasting

Python automatically converts types when needed (e.g., int + float = float).

Code

```
num1 = 10 # int

num2 = 20.5 # float

result = num1 + num2 # Python promotes int to float

print(result)

print(type(result))
```

Output

```
30.5

<class 'float'>
```

2. Explicit Typecasting

Manually convert using built-in functions.

Function	Converts to
`int()`	integer
`float()`	floating-point number
`str()`	string
`bool()`	boolean (True/False)
`list()`	list
`tuple()`	tuple
`set()`	set
`dict()`	dictionary

Code

```
a = 30.5

b = int(a) # truncates decimal part

print(b)

print(type(b))
```

Output

```
30

<class 'int'>
```

13 Operators

Operators are symbols that perform operations on variables and values.

13.1 Arithmetic Operators

Op	Operation	Example	Result
+	Addition	`9 + 2`	11
-	Subtraction	`9 - 2`	7
*	Multiplication	`9 * 2`	18
/	Division	`9 / 2`	4.5
//	Floor Division	`9 // 2`	4
%	Modulus (remainder)	`9 % 2`	1
**	Exponentiation	`9 ** 2`	81

Code

```
print(9 + 2) # 11

print(9 - 2) # 7

print(9 * 2) # 18

print(9 / 2) # 4.5

print(9 // 2) # 4

print(9 % 2) # 1

print(9 ** 2) # 81
```

Output

```
11

7

18

4.5

4

1

81
```

13.2 Assignment Operators

Op	Description	Same as
=	Assign	a = 5
+=	Add and assign	a = a + 5
-=	Subtract and assign	a = a - 5
*=	Multiply and assign	a = a * 5
/=	Divide and assign	a = a / 5
//=	Floor divide and assign	a = a // 5
%=	Modulus and assign	a = a % 5
**=	Exponentiate and assign	a = a ** 5

Code

```
a = 10

a += 5 # a = 15
```

```
print(a)

a -= 2 # a = 13

print(a)

a *= 2 # a = 26

print(a)
```

Output

```
15

13

26
```

13.3 Comparison Operators

Op	Meaning	Example	Result
>	Greater than	`10 > 5`	`True`
<	Less than	`10 < 5`	`False`
>=	Greater than or equal	`10 >= 10`	`True`
<=	Less than or equal	`10 <= 5`	`False`
==	Equal to	`10 == 5`	`False`
!=	Not equal to	`10 != 5`	`True`

13.4 Logical Operators

Op	Description
`and`	Returns True if BOTH statements are true
`or`	Returns True if AT LEAST ONE statement is true
`not`	Reverses the result (True becomes False, False becomes True)

Precedence: not > and > or

not is evaluated first, then and, then or

Code

```
a, b = 10, 5

print(a > b and a < 20) # True and True = True

print(a > b or a < 3) # True or False = True
```

```
print(not(a > b)) # not True = False


has_good_credit = True

has_pending_loan = False

is_approved = has_good_credit and not has_pending_loan

print("Loan Approved:", is_approved)
```

Output

```
True

True

False

Loan Approved: True
```

13.5 Membership Operators

Operator	Description
`in`	Returns True if value is found in the sequence
`not in`	Returns True if value is NOT found in the sequence

Code

```
s = "Darshan"

print("d" in s) # False (case-sensitive)

print("D" in s) # True

print("shan" in s) # True

print("xyz" not in s) # True
```

Output

```
False

True

True

True
```

13.6 Identity Operators

Operator	Description
`is`	Returns True if both variables point to the SAME object in memory

`is not`	Returns True if both variables point to DIFFERENT objects in memory

Code

```
a = 10

b = 10

print(a is b) # True (Python caches small integers)

print(a is not b) # False
```

Output

```
True

False
```

14 Memory & id() Function

What is id()?

id() returns the memory address of an object. Useful for debugging and understanding how Python manages memory.

Python caches small integers (-5 to 256) and short strings, so identical values share the same memory address.

Code

```
a = 100

b = 150

print(id(a)) # different addresses

print(id(b))
```

Output

```
4308585184

4308586784
```

Integer caching (small integers share addresses):

Code

```
x = 50

y = 50

print(id(x) == id(y)) # True - same object!

y = y + 2
```

```
print(id(x) == id(y)) # False - different object now
```

Output

```
True

False
```

String interning (short strings share addresses):

Code

```
p = "Darshan"

q = "Darshan"

print(id(p) == id(q)) # True - same interned string
```

Output

```
True
```

15 To-Do Tasks

Task 1: String to Integer Conversion

A client enters the number of units as "1200" (string). Convert it to integer and calculate total cost at 45.5 USD per unit.

Hint: Use int() to convert the string input.

Task 2: Float to Integer Conversion

Amazon records a purchase value as 25340.75. Convert it to integer to represent the rounded-down invoice value. Print both original and converted values.

Hint: Use int() on the float value.

Task 3: Using Arithmetic Operators

Two clients ordered 300 and 450 units respectively. - Find total units (addition) - Difference in units (subtraction) - Multiply total units by price per unit 25 - Divide order 1 units by 12 (carton packing check)

*Hint: Use +, -, *, /.*

Task 4: Comparison Operators

Client A ordered 1500 units, Client B ordered 1200. - Verify who ordered more. - Check if both ordered equal units.

Hint: Use >, <, ==.

Task 5: Logical Operators with Conditions

A client qualifies for a special discount if they ordered more than 1000 units AND are a premium member. Check if "TechSupply" with 1200 units and premium membership qualifies.

Hint: Use and, or operators.

Happy Learning!

Python Basics . 01_Python_Basics.ipynb . Premium Study Notes